SOFTWARE ENGINEERING · AI SYSTEMS

# Agentic Harness для написания кода

Как LLM превращается в управляемого coding agent

ПРОБЛЕМА

# LLM сама по себе не умеет надежно менять кодовую базу

### Chat Completion

- Одноразовый ответ
- Без доступа к файлам
- Без проверки результата
- Без памяти о контексте
- Без инструментов

### Agentic Coding System

- ✓ Контекст + инструменты
- ✓ Планирование и декомпозиция
- ✓ Валидация и тесты
- ✓ Память и состояние
- ✓ Обратная связь и ремонт

ОПРЕДЕЛЕНИЕ

# Agentic Harness — управляющая обвязка вокруг модели

Harness превращает модель в исполнителя задач: планирует, читает файлы, редактирует код, запускает тесты, анализирует ошибки.

```
// harness = execution + control layer
class CodingHarness {
  plan(task) → steps[]
  execute(step) → result
  validate(result) → pass | fail
  repair(failure) → patch
}
```

LLM
reasoning core

→

Harness
execution / control

1. Принимает задачу от пользователя
2. Строит контекст из репозитория
3. Генерирует план через LLM
4. Выполняет шаги инструментами
5. Проверяет и ремонтирует

АРХИТЕКТУРА

# Высокоуровневая архитектура агентской системы

User Request

Orchestrator
control loop

LLM / Reasoning
engine

Context Manager

Repository Indexer

Tool Executor

Patch Manager

Shell / Sandbox

Test Runner

Memory / State

Policy & Permissions

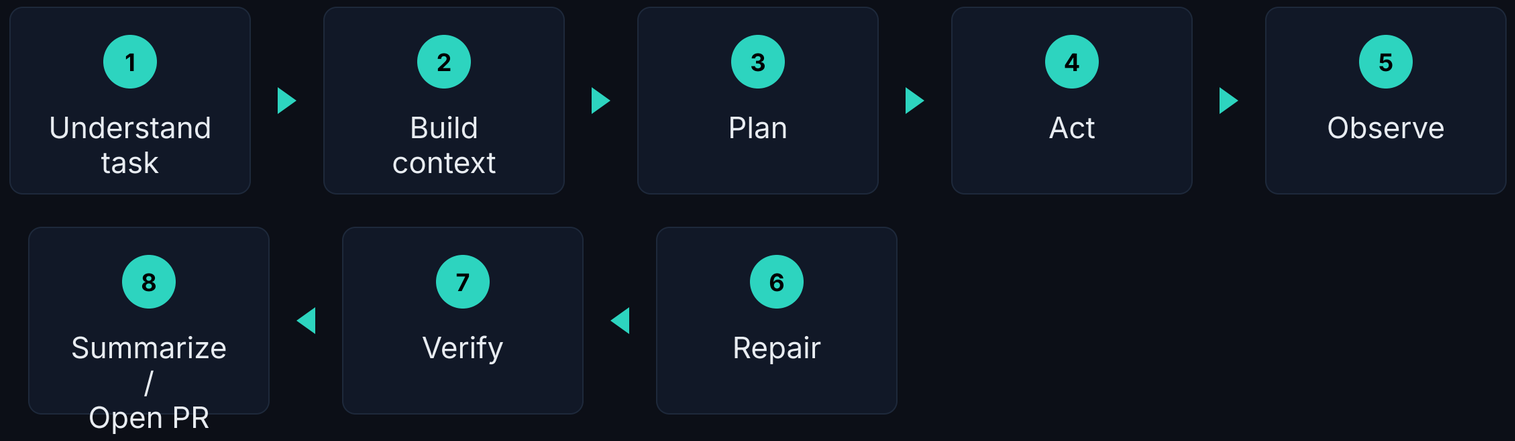
Telemetry / Evals

Все компоненты связаны через orchestrator, который управляет жизненным циклом задачи

ЖИЗНЕННЫЙ ЦИКЛ

# Главный цикл работы агента

Надежность появляется из итеративного цикла, а не из одного ответа модели




```
// control loop псевдокод
while (!task.complete) {
  context = buildContext(task, history);
  plan = llm.plan(context);
  result = execute(plan);
  if (!validate(result)) { repair(result.error); }
  else { task.complete = true; }
}
```

КОНТЕКСТ

# Как агент понимает кодовую базу

Качество контекста часто важнее размера модели

### Чтение файлов

Прямой доступ к исходникам, конфигам и документации

### Repo Map

Структура директорий и связи между модулями

### Semantic Search

Векторный поиск по codebase для релевантных фрагментов

### Dependency Graph

Импорты, вызовы функций и иерархия классов

### Symbol Index

AST-индекс классов, функций и переменных

### Issue / PR Context

Описание задачи, комментарии и история изменений

ИНСТРУМЕНТАРИЙ

# Инструменты агента

Инструменты должны быть явными, логируемыми и ограниченными

📖
**read_file**
Чтение содержимого

🔍
**search**
Поиск по коду

✏️
**edit_file**
Редактирование

🖥️
**run_shell**
Команды shell

🧪
**run_tests**
Запуск тестов

📜
**inspect_logs**
Просмотр логов

📦
**package_mgr**
Зависимости

🔧
**linter**
Линтинг / Types

🌐
**browser / API / docs lookup**
Внешние источники (если разрешено)

Принцип: Каждый вызов инструмента — это явное действие с входными параметрами, выходом и логом. Агент не должен иметь неограниченного доступа.

ПЛАНИРОВАНИЕ

# Декомпозиция задачи

Harness помогает разбить сложную задачу на управляемые шаги

## Этапы планирования

1. Intent extraction — что именно нужно
2. Constraints — ограничения и требования
3. Affected modules — затронутые модули
4. Implementation plan — план реализации
5. Risk analysis — анализ рисков
6. Rollback strategy — стратегия отката

## Пример: «Добавить OAuth login»

```
// План декомпозиции
1. schema — добавить таблицу users/oauth
2. backend — роут /auth/callback
3. frontend — кнопка login + state
4. tests — unit + integration
5. docs — update README.md
6. rollback — миграция обратима
```

Каждый шаг = один коммит или один инструментальный вызов

# Редактирование кода

Агент должен вносить маленькие проверяемые изменения

1. Выбрать файлы
2. Сгенерировать patch
3. Применить diff
4. Проверить конфликт
5. Перечитать изменённые участки
6. Минимизировать scope

### Хорошо

- ✓ Изменение ≤ 50 строк
- ✓ Один фокус на коммит
- ✓ Тесты проходят сразу
- ✓ Diff читается человеком

### Плохо

- — 500+ строк без разбивки
- — Рефакторинг + фича в одном
- — Сломанные тесты игнорируются
- — Неочевидные site-effects

ИЗОЛЯЦИЯ

# Sandbox и execution environment

Coding agent должен быть мощным, но не безграничным

### 🔒 Изоляция команд

Каждая команда выполняется в отдельном контейнере или VM

### ⏱️ Лимиты ресурсов

Таймауты CPU, RAM и время жизни сессии

### 🚫 Запрет опасных операций

rm -rf /, форматирование дисков, raw network

### ✅ Allowlist / Denylist

Явный список разрешенных команд и путей

### 🔐 Работа с секретами

Env-переменные, vault, никаких хардкодов

### 🌐 Network Policy

Ограничение исходящих соединений по белому списку

Sandbox — это safety boundary, а не помеха продуктивности

ВАЛИДАЦИЯ

# Воронка проверки результата

Harness должен не только писать код, но и доказывать, что изменение работает

Format ▶ Lint ▶ Typecheck ▶ Unit Tests ▶ Integration ▶ Regression

▼

**Manual Reasoning Review**
LLM оценивает корректность логики

**100%**
Форматирование пройдено

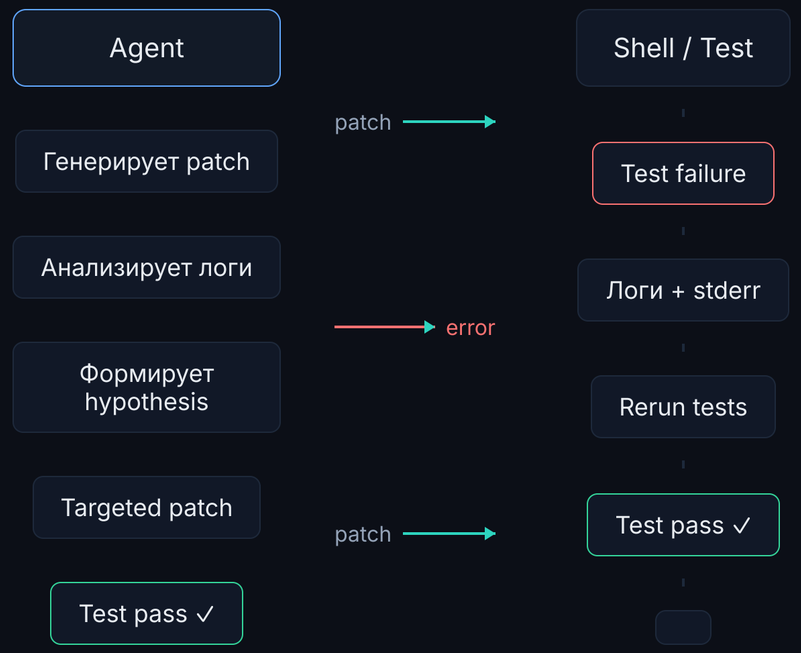
**0**
Ошибок линтера

✓
Все тесты green

SELF-REPAIR

# Ошибки и самовосстановление

Типичный цикл: test failure → log analysis → hypothesis → targeted patch → rerun



БЕЗОПАСНОСТЬ

# Безопасность и governance

Harness — это also safety boundary

### 🔑 Permission Gates

Явные разрешения на каждый класс операций

### 👤 Human Approval

Destructive операции требуют подтверждения

### 🔒 Secrets & Credentials

Никаких токенов в логах или промптах

### 💥 Destructive Commands

Защита от rm, drop, truncate и аналогов

### 📦 Supply-Chain Risk

Проверка зависимостей перед установкой

### 📋 Audit Log

Полная трассировка действий агента

Важно: Policy enforcement должен быть deterministic и не зависеть от решения LLM.

КАЧЕСТВО

# Telemetry, evals и метрики

Без evals агентская система не улучшается управляемо

| | | | |
|---|---|---|---|
| **85%** Task Success Rate | **94%** Tests Passed | **3.2** Iterations Avg | **12%** Human Intervention |
| **42** Patch Size (lines) | **2%** Regression Rate | **8m** Time-to-PR | **$0.12** Cost per Task |

Метрики собираются на каждой итерации и анализируются для улучшения промптов, инструментов и политик

ИТОГ

# 5 принципов хорошего coding harness

### 1. Explicit Tools

Каждое действие агента — через явный, логируемый и ограниченный интерфейс

### 2. Tight Feedback Loop

План → действие → наблюдение → коррекция за секунды, а не минуты

### 3. Scoped Context

Только релевантный контекст: файлы, символы, зависимости, история изменений

### 4. Safe Execution

Sandbox, allowlist, audit log и human-in-the-loop для опасных операций

### 5. Measurable Quality

Success rate, time-to-PR, cost, regressions — без метрик нет улучшений

**LLM генерирует гипотезы; harness превращает их в проверяемые изменения кода**